

PMSCS 622P

Programming Languages

Lecture 08: Classes and Objects

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

Lecture Roadmap

- 1 **The Paradigm Shift:** Procedural vs. Object-Oriented.
- 2 **Core Concepts:** Blueprints and Instances.
- 3 **Syntax & Semantics:** Defining Custom Data Types.
- 4 **Memory Mechanics:** The Stack, The Heap, and References.
- 5 **Encapsulation:** Protecting the Invariant (Public vs Private).
- 6 **Object Lifecycle:** Constructors and Initialization.
- 7 **The Identity Problem:** Understanding `this` and `self`.
- 8 **Practice Pattern:** Decomposing Real-World Entities.

1.1 The Problem with Procedural Programming

So far, we have written programs using **Procedural Programming** (Functions calling Functions).

The Disconnect

In Procedural code, **Data** (variables) and **Behaviors** (functions) are separate.

- We have lists of 'student_names', 'student_ids', 'student_gpas'.
- We pass these lists to functions like 'calculateGPA(ids, gpas)'.

The Flaw: As programs grow to millions of lines, keeping track of which function is allowed to modify which piece of scattered data becomes a nightmare.

1.2 Enter Object-Oriented Programming (OOP)

Object-Oriented Programming is a **mental model** for writing code.

Instead of asking: *"What sequential steps do I need to perform?"*

We ask: **"What entities exist in my system, and how do they interact?"**

- **Procedural:** `withdraw(account_balance, 50)`

- **OOP:** `myAccount.withdraw(50)`

In OOP, the data itself knows how to perform operations.

1.3 State and Behavior

An "Object" is a software bundle that combines two things:

1. State (Attributes / Data)

- What the object *knows* or *is*.
- Represented by Variables.
- e.g., A Dog knows its name, breed, age, color.

2. Behavior (Methods / Logic)

- What the object *can do*.
- Represented by Functions inside the object.
- e.g., A Dog can bark(), eat(), sleep().

2.1 Class vs. Object

The most important distinction in OOP:

The Class (The Blueprint)

A **Class** is a blueprint, template, or design. It does not exist physically in memory as data. It just defines *what* an object will look like.

Example: The architectural blueprint for a house.

The Object (The Instance)

An **Object** is a specific, physical realization of the class. It occupies RAM. You can have many objects built from one class.

Example: The actual physical house built at 123 Main St.

2.2 A Custom Data Type

Fundamentally, creating a Class is creating a **Custom Data Type**.

- Languages give us primitives: `int`, `float`, `boolean`.
- But what if we need a `Student` type? Or a `BankAccount` type? Or a `Monster` type?

A Class tells the compiler: *"Hey, whenever I declare a variable of type `Student`, I want you to allocate enough memory to hold a string (Name), an integer (ID), and a float (GPA) all bundled together."*

3.1 Defining a Class

Let's define a simple Dog class with one attribute (name) and one method (bark).

C++

```
class Dog {  
public:  
    string name;  
  
    void bark() {  
        cout << "Woof!";  
    }  
}; // Semicolon required!
```

Java

```
class Dog {  
    public String name;  
  
    public void bark() {  
        System.out.println  
            ("Woof!");  
    }  
}
```

Python

```
class Dog:  
    # State defined in  
    constructor  
    def __init__(self, n):  
        self.name = n  
  
    def bark(self):  
        print("Woof!")
```


3.2 Instantiation (Creating the Object)

How do we bring the blueprint to life? This is called **Instantiation**.

C++

```
// Created on the Stack  
Dog myDog;  
myDog.name = "Rex";  
myDog.bark();
```

Java

```
// Created on the Heap  
Dog myDog = new Dog();  
myDog.name = "Rex";  
myDog.bark();
```

Python

```
# Created on the Heap  
my_dog = Dog("Rex")  
  
my_dog.bark()
```

Notice the Dot Operator (.). We use it to reach inside the object to access its state or behavior.

4.1 The Stack vs. The Heap

To truly understand objects, we must look at RAM. Computer memory is divided into two main areas for our programs:

The Stack

- Fast, structured, small.
- Holds local variables and function calls (Activation Records).
- Cleans itself up automatically when a function ends.

The Heap

- Massive, unstructured, slower.
- Used for dynamic memory (things whose size or lifespan we don't know at compile time).
- **This is where Objects live in Java and Python.**

4.2 The Remote Control Analogy

When you write `Dog d1 = new Dog();` in Java (or `d1 = Dog()` in Python), what is `d1`?

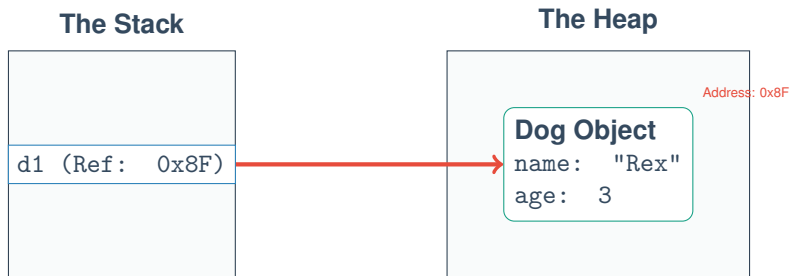
Crucial Concept

`d1` is **NOT** the Dog. `d1` is a **Reference Variable** (a pointer).

Think of `d1` as a *Remote Control*. The actual Dog object is a TV sitting far away in the Heap memory.

- The Remote Control (`d1`) lives on the **Stack**.
- The TV (The Object Data) lives on the **Heap**.
- The dot operator (`d1.bark()`) is you pressing a button on the remote.

4.3 Visualizing Object Memory



The variable just holds the memory address of the object.

4.4 The Danger of Aliasing

What happens if we copy a reference?

```
Dog d1 = new Dog();  
d1.name = "Rex";  
  
Dog d2 = d1;           // What did we just copy?  
d2.name = "Max";  
  
System.out.println(d1.name); // Output: ?
```

- We did **not** copy the Dog! We copied the *Remote Control*.
- Now we have two remotes pointing to the exact same TV.
- Changing the channel with d2 changes the TV for d1.
- **Output: Max.**

4.5 C++ is Different (Pass by Value Default)

C++ Memory Mechanics

In C++, if you write `Dog d1;`, the object is created **directly on the Stack**. If you write `Dog d2 = d1;`, C++ makes a **complete physical clone** of the object.

To get Java/Python behavior (Heap allocation) in C++, you must use pointers and the `new` keyword manually:

```
Dog* d1 = new Dog(); // d1 is now a remote control.
```

5.1 What is Encapsulation?

Encapsulation is the practice of bundling data and methods into a single unit (the class), and **restricting direct access** to some of the object's components.

- Why? To protect the **Object Invariant**.
- An "Invariant" is a rule that must always be true for the object to be valid.
- Example: A Bank Account's balance cannot be manipulated directly; otherwise, someone could set it to -999999 or "Potato".

5.2 Access Modifiers

We use **Access Modifiers** to enforce Encapsulation.

Public (`public`)

- Anyone from outside the class can read/modify this.
- Usually reserved for Methods (Behaviors).

Private (`private`)

- Only the class itself can see or touch this.
- Usually used for State (Variables).

5.3 Getters and Setters

If data is private, how do we use it? We provide public "Gatekeeper" methods.

```
class BankAccount {  
    private double balance = 0; // Hidden!  
  
    // Getter (Read-only access)  
    public double getBalance() {  
        return balance;  
    }  
  
    // Setter (Write access, with rules!)  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
}
```

5.4 Why not just make it public?

The Vending Machine Analogy.

- **Public Data:** The vending machine is an open box. You put money in, and you grab the chips. Someone could just take the chips without paying.
- **Private Data + Methods:** The vending machine is locked glass. You interact with the *interface* (buttons, coin slot). The machine enforces the rules (checks payment, dispenses item).

Encapsulation prevents other programmers from breaking your objects.

5.5 Encapsulation in Python

Python does not have strict 'public' and 'private' keywords enforced by the compiler.

Instead, Python uses a **naming convention**:

```
class BankAccount:
    def __init__(self):
        # Prefix with double underscore makes it "hidden" (name mangling)
        self.__balance = 0

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

Philosophy: "We are all consenting adults here." Python trusts programmers not to touch `__hidden` variables.

6.1 The Birth of an Object

When we write `new Dog()`, we are doing two things:

- 1 `new`: Asks the OS for raw memory on the Heap.
- 2 `Dog()`: Calls a special function called the **Constructor**.

What is a Constructor?

A constructor is a special method invoked automatically the moment an object is created. Its sole purpose is to **initialize the state** (set starting variable values) so the object is born in a valid state.

6.2 Constructor Syntax (C++ / Java)

In C++ and Java, the constructor:

- Has the **exact same name** as the class.
- Has **no return type** (not even 'void').

```
class Student {  
    private String name;  
  
    // Parameterized Constructor  
    public Student(String startingName) {  
        name = startingName;  
        System.out.println("A student is born!");  
    }  
}  
  
// Usage:  
Student s1 = new Student("Alice"); // Forces us to provide a name!
```

6.3 Constructor Syntax (Python)

In Python, the constructor is always named `__init__` (initialize).

```
class Student:
    # The __init__ method is the constructor
    def __init__(self, starting_name):
        # We define and initialize attributes here
        self.name = starting_name
        print("A student is born!")

# Usage:
s1 = Student("Alice")
```

Note: We do not need to declare variables before `__init__` in Python; assigning to `self.variable` creates them dynamically.

6.4 Default Constructors

What if you don't write a constructor?

- The compiler (C++/Java) provides a **Default Constructor** behind the scenes.
- It takes no arguments: `Student()`
- It sets everything to "zero" values (0 for ints, `null` for objects, `false` for booleans).

Warning!

If you write *any* custom constructor (e.g., `Student(String name)`), the compiler takes away the free default one. You can no longer say `new Student()` unless you explicitly write a no-argument constructor yourself.

6.5 The Death of an Object (Destructors)

What happens when we are done with an object?

C++ (Manual Memory Management):

- You must explicitly delete Heap objects: `delete myDog;`
- This triggers a **Destructor** method (e.g., `~Dog()`) to clean up resources.

Java / Python (Garbage Collection):

- The JVM/Interpreter monitors the memory.
- When all "Remote Controls" (references) to an object are gone (e.g., the function ends), the object is considered "unreachable."
- The **Garbage Collector** automatically deletes it to free up RAM. You don't write destructors.

7.1 How do methods know which data to use?

Imagine we have two objects:

```
Dog d1 = new Dog("Rex");  
Dog d2 = new Dog("Max");
```

When we call `d1.bark()`, it prints "Rex says Woof!".

When we call `d2.bark()`, it prints "Max says Woof!".

The Conceptual Mystery

There is only **one** copy of the `bark()` code in memory (in the Class blueprint). How does that single block of code know whether to use Rex's memory or Max's memory?

7.2 The Hidden Parameter

The compiler pulls a trick. Every time you call a method on an object, the compiler secretly passes the **object's memory address** as a hidden parameter to the function.

What you write:

```
| d1.bark();
```

What the computer actually executes:

```
| Dog_bark(d1); // Pass the remote control as an argument!
```

7.3 this (C++/Java) and self (Python)

Inside the method, we have access to this hidden parameter.

- **Python** makes this explicit. You MUST write `self` as the first parameter.
- **C++ / Java** makes this implicit. The keyword `this` is always available in the background.

Python

```
def bark(self):  
    print(self.name + " says Woof!")  
    # self is d1
```

Java

```
public void bark() {  
    System.out.println(this.name + " says  
        Woof!");  
    // 'this' is implied, but you can  
        write it.  
}
```

7.4 Using `this` to resolve ambiguity

The most common use of `this` in Java/C++ is when parameter names clash with attribute names.

```
class Student {  
    private String name; // Attribute  
  
    public Student(String name) { // Parameter  
        // name = name; // Bug! Assigns parameter to itself.  
  
        // Fix: Use 'this' to point to the Object's attribute.  
        this.name = name;  
    }  
}
```

8.1 Modeling Reality

OOP is about mapping real-world nouns to Classes, and verbs to Methods.

Task: Design a Class for a Digital Wallet.

1 State (Data): What does a Wallet have?

- An owner's name.
- A balance (amount of money).
- A currency type (USD, BDT).

2 Behavior (Methods): What can a Wallet do?

- Add money (Deposit).
- Send money (Transfer).
- Check balance.

8.2 Defining the Rules (Invariants)

Before coding, define the rules your Class must protect:

- **Rule 1:** A wallet cannot have a negative balance.
- **Rule 2:** You cannot send more money than you have.
- **Rule 3:** The owner's name is set at birth (constructor) and cannot be changed later.

These rules dictate our Encapsulation strategy. 'balance' must be private. 'owner' must have no setter method.

8.3 Solution: The Wallet Class (Java)

```
public class Wallet {
    private String owner;           // Private: No setter, set in constructor
    private double balance;         // Private: Protected by rules

    public Wallet(String ownerName, double initialDeposit) {
        this.owner = ownerName;
        this.balance = initialDeposit > 0 ? initialDeposit : 0;
    }

    public boolean sendMoney(double amount) {
        if (amount > 0 && this.balance >= amount) {
            this.balance -= amount;
            return true; // Success
        }
        return false; // Failed rule
    }

    public double getBalance() { return this.balance; }
}
```

8.4 Practice Problem: The Library Book

Task: Decompose a `Book` for a Library System.

Requirements:

- Must track Title, Author, and whether it is currently checked out.
- A user should be able to `borrow()` the book.
- A user should be able to `return()` the book.
- You cannot borrow a book that is already checked out.

Think: What are the states? What are the methods? What are the invariants?

8.5 Solution: The Book Class (Python)

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.__is_checked_out = False # Private, starts available

    def borrow_book(self):
        if self.__is_checked_out:
            print("Sorry, book is unavailable.")
        else:
            self.__is_checked_out = True
            print(f"You borrowed {self.title}.")

    def return_book(self):
        self.__is_checked_out = False
        print(f"{self.title} returned.")
```

8.6 Objects Interacting with Objects

The true power of OOP is objects passing messages to other objects.

```
class Player {
    private String name;

    // A method taking ANOTHER Object as an argument
    public void attack(Monster target) {
        System.out.println(this.name + " attacks!");
        target.takeDamage(10); // Sending a message to the Monster
    }
}

// Main:
Player p1 = new Player("Hero");
Monster m1 = new Monster("Goblin");
p1.attack(m1);
```

Summary: The Object Model

- 1 **Classes** are blueprints (Custom Data Types). **Objects** are physical instances in memory.
- 2 Variables holding objects are **References** (remote controls). The data lives on the **Heap**.
- 3 **Encapsulation** (Private/Public) protects your object's internal rules.
- 4 **Constructors** guarantee an object is born in a valid state.
- 5 The `this` / `self` pointer is how an object identifies its own data in memory.

Common Beginner Errors

1. Forgetting to Instantiate

Declaring `Dog d1;` and calling `d1.bark()` without `new Dog()`. Result: `NullPointerException`.

2. Confusing Class vs Instance Variables

Assigning a value to the Class itself instead of the Object (e.g., `Dog.name = "Rex"` instead of `myDog.name`).

3. Accidental Aliasing

Setting `objA = objB` and expecting them to be independent copies. They are now the exact same object in memory!